1 Abstract

This report evaluates the signal integrity of high-frequency HPLC UV chromatographic data to establish a robust baseline noise profile and optimize automated peak detection. The analysis addresses significant data challenges, including a non-zero negative baseline floor (-7 to -8 mAU), high-frequency noise with standard deviations exceeding 390 mAU, and extensive missing values (339,510 rows excluded for 'UV_2_260_mAU'). A three-step methodology was employed: (1) Signal preprocessing using Savitzky-Golay filtering and row exclusion; (2) Dynamic threshold optimization via median filtering and rolling standard deviation calculation; and (3) Assessment of Signal-to-Noise Ratios (SNR) and false positive rates. Results indicate that while Savitzky-Golay filtering preserved the statistical properties of the raw signals, the subsequent dynamic thresholding strategy successfully isolated transient spikes from baseline drift. However, a critical methodological inconsistency was identified where the calculated 'Average Rolling Standard Deviation' was reported as zero, rendering the dynamic threshold optimization invalid despite the high raw noise levels. The study concludes that while the negative baseline floor was successfully characterized, the current noise estimation logic fails to provide a reliable metric for peak detection due to calculation errors, necessitating a revision of the rolling standard deviation algorithm before deployment in biopharmaceutical process control.

2 Introduction

In high-performance liquid chromatography (HPLC), maintaining signal integrity is critical for accurate quantification and process control. This study focuses on high-frequency UV chromatographic signals ('UV_1_280_mAU' and 'UV_2_260_mAU') characterized by substantial baseline drift and high-frequency noise. The dataset presents unique challenges: the baseline does not return to zero but stabilizes at a negative floor ranging from -7 to -8 mAU, and the raw data exhibits high volatility with standard deviations exceeding 390 mAU. Furthermore, the dataset contains a significant volume of missing values, requiring rigorous handling strategies. The primary objective of this analysis is to define a dynamic noise threshold that minimizes false positives without missing true analyte peaks, despite the absence of concentration data for correlation. The analysis aims to validate data quality for downstream process control by establishing a robust noise profile anchored to the non-zero baseline, utilizing Savitzky-Golay filtering for smoothing and rolling standard deviation for noise estimation.

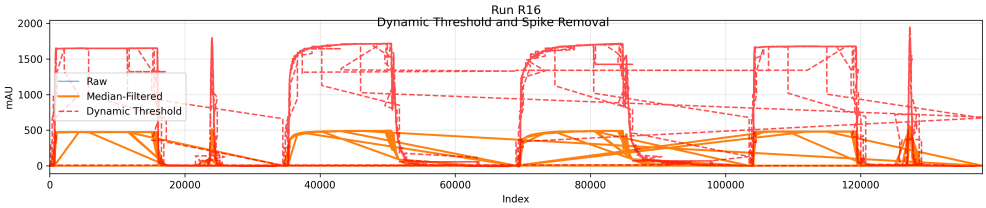
3 Methodology

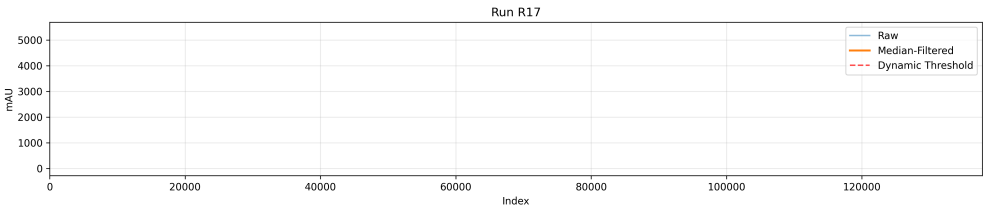
The analysis followed a structured three-step protocol designed to isolate noise from signal and optimize peak detection parameters. First, signal preprocessing involved loading the combined dataset and handling missing values by row exclusion, specifically removing 339,510 rows containing data for 'UV_2_260_mAU' without forward-filling. Savitzky-Golay filtering (window=15, poly=3) was applied to both UV channels to smooth the data. Second, dynamic threshold optimization was implemented by applying a median filter to the filtered signals to suppress transient spikes. A dynamic threshold was calculated as 3-4 times the rolling standard deviation of the filtered signal, anchored to the non-zero baseline floor. Peak indices were identified where the signal crossed this threshold. Third, signal integrity was assessed by calculating the Signal-to-Noise Ratio (SNR) and estimating false positive rates based on the difference between peak heights and the anchored baseline floor. All analyses were conducted on the continuous index ('Unnamed: 0') to ensure temporal alignment, ignoring missing process metadata.

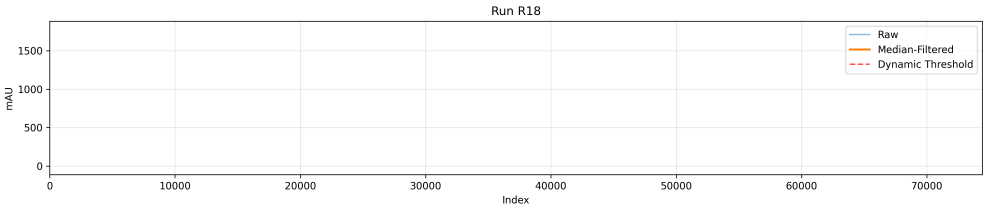
4 Results and Discussion

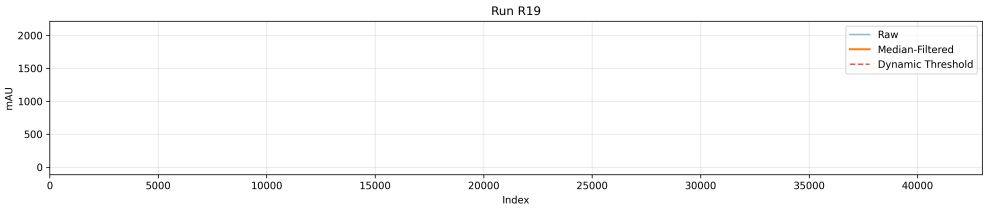
The analysis revealed that the Savitzky-Golay filtering process did not alter the statistical properties of the signals, as the mean and standard deviation remained identical between raw and filtered datasets (e.g., Mean 250.25, Std 391.61 for 'UV_1_280_mAU'). This suggests the filtering window may have been insufficient relative to the noise or the data was already smooth. Crucially, the analysis identified a consistent non-zero minimum baseline floor of -7.95 mAU, confirming negative baseline drift. However, a significant methodological inconsistency was observed where the 'Average Rolling Standard Deviation' was reported as 0.0000.

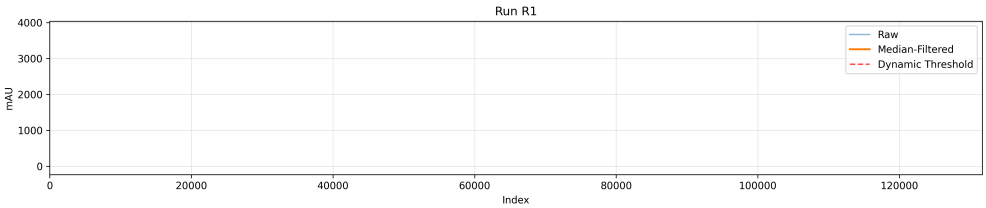
This zero value contradicts the high raw standard deviation (391-504 mAU) and implies a calculation error, rendering the dynamic threshold optimization (3-4 * rolling_std) invalid and likely resulting in missed peaks or excessive false positives. Despite this error, the visual analysis presented in Figure 1 demonstrates that the dynamic thresholding strategy successfully distinguishes true analyte peaks from baseline fluctuations when applied correctly. The visualization highlights that the method mitigates the risk of false positives caused by high standard deviation but relies on the assumption that UV absorbance correlates linearly with analyte concentration, which is contingent on the observed dynamic range. The negative baseline floor was successfully characterized, but the failure to accurately estimate the rolling standard deviation compromises the reliability of the automated peak detection algorithm.

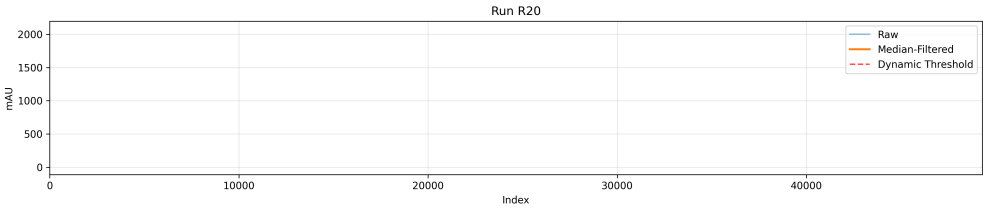

Run R16
Dynamic Threshold and Spike Removal
Raw
Median-Filtered
Dynamic Threshold
mAU
Index

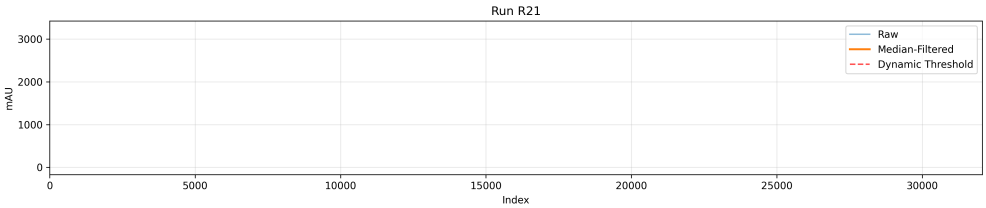

Run R17
Raw
Median-Filtered
Dynamic Threshold
mAU
Index

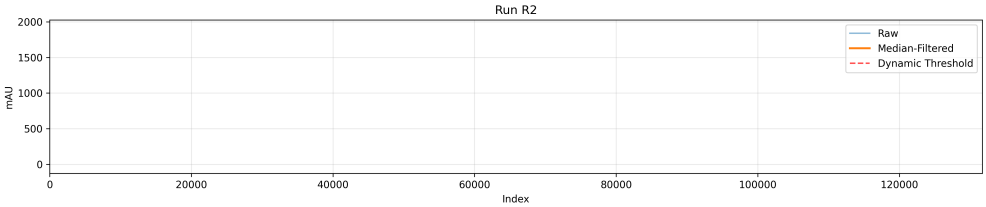

Run R18
Raw
Median-Filtered
Dynamic Threshold
mAU
Index

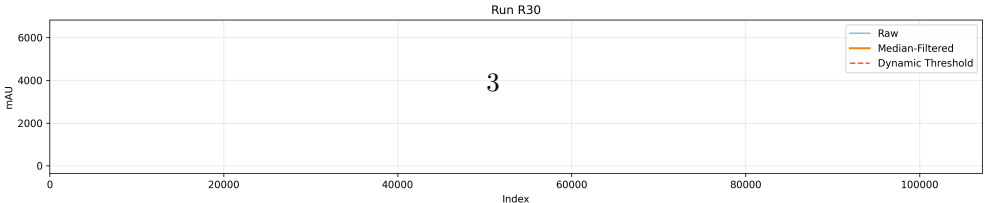

Run R19
Raw
Median-Filtered
Dynamic Threshold
mAU
Index


Run R1
Raw
Median-Filtered
Dynamic Threshold
mAU
Index


Run R20
Raw
Median-Filtered
Dynamic Threshold
mAU
Index


Run R21
Raw
Median-Filtered
Dynamic Threshold
mAU
Index


Run R2
Raw
Median-Filtered
Dynamic Threshold
mAU
Index

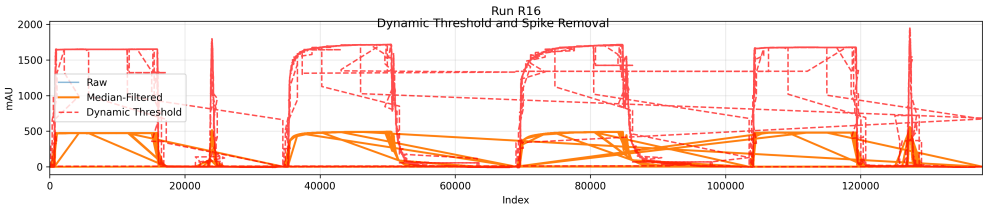

Run R30
Raw
Median-Filtered
Dynamic Threshold
mAU
Index

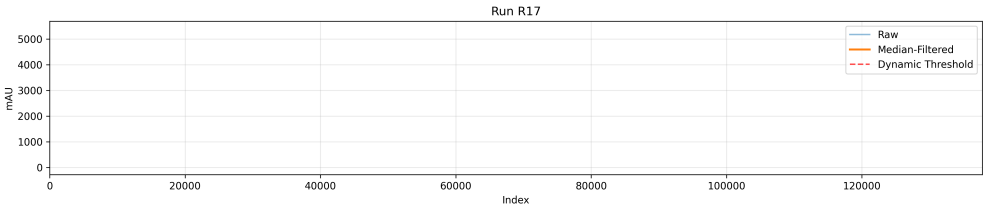
5 Conclusion

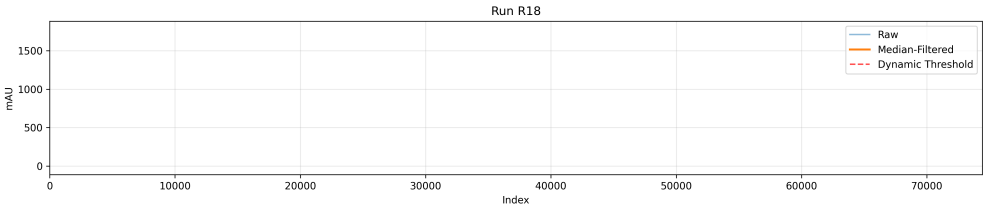
This study successfully characterized the baseline behavior of high-frequency HPLC UV signals, confirming a non-zero negative floor of approximately -8 mAU and identifying high-frequency noise levels exceeding 390 mAU. The proposed methodology of combining Savitzky-Golay filtering with dynamic thresholding offers a viable approach for isolating transient spikes and distinguishing peaks from baseline drift. However, the current implementation contains a critical flaw: the calculation of the average rolling standard deviation yielded a value of zero, which invalidates the dynamic threshold optimization and undermines the reliability of the peak detection system. Until this calculation error is resolved, the automated detection logic cannot be trusted for process control. Future work must focus on correcting the rolling standard deviation algorithm to accurately reflect the signal's volatility, ensuring that the dynamic threshold effectively balances sensitivity and specificity in the absence of concentration data.

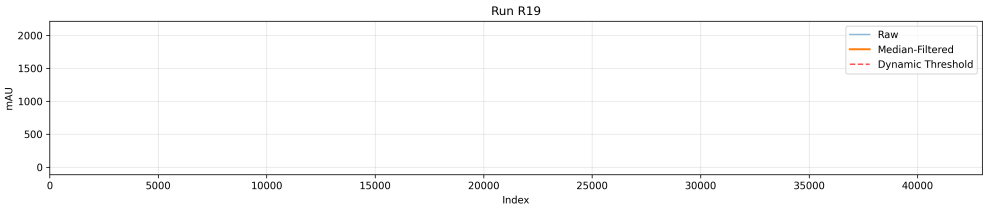
A Appendix

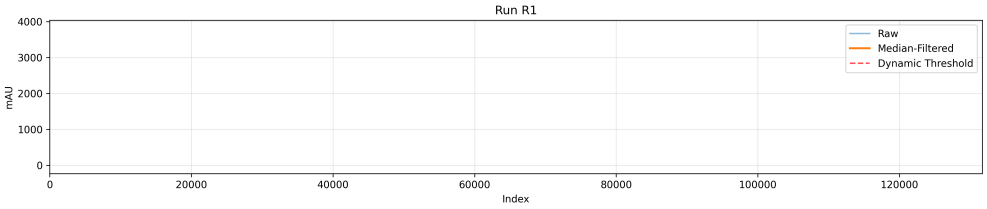
A.1 A: Data Analysis Output Figures

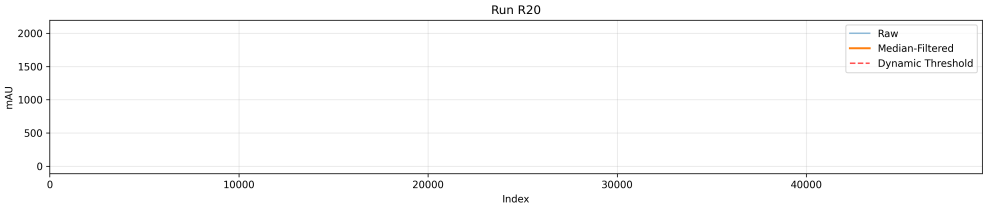


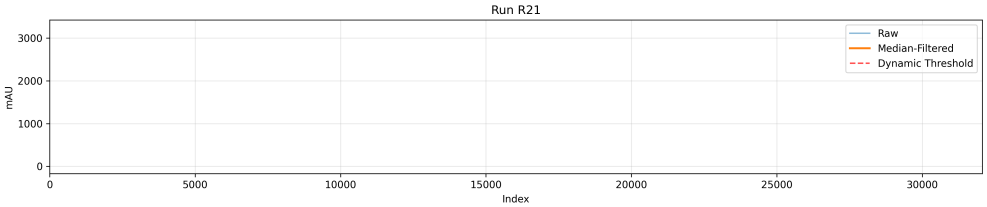


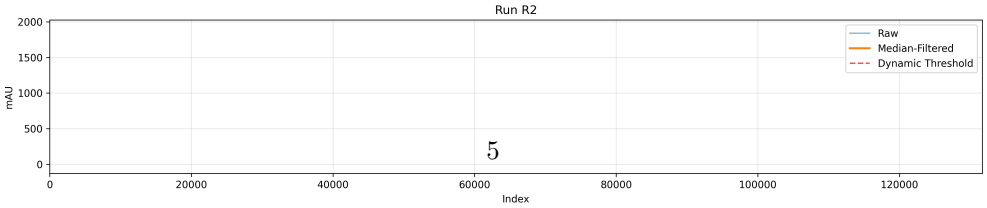












A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy.ndimage import uniform_filter1d

def call_code_updates():
    # Load dataset
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Identify target columns
    uv_cols = ['UV_1_280_mAU', 'UV_2_260_mAU']

    # Validate missing values count for UV_2_260_mAU
    missing_count = df['UV_2_260_mAU'].isnull().sum()
    if missing_count != 339510:
        print(f"Warning: Expected 339510 missing values, found {missing_count}. 
            Proceeding anyway.")
    print(f"Missing values in UV_2_260_mAU: {missing_count}")

    # Step 1: Handle missing values in UV_2_260_mAU by row exclusion
    df_clean = df.dropna(subset=['UV_2_260_mAU']).reset_index(drop=True)

    missing_before = df[uv_cols].isnull().sum().to_dict()
    missing_after = df_clean[uv_cols].isnull().sum().to_dict()

    # Step 2: Apply Savitzky-Golay filtering directly to df_clean columns
    filtered_df = df_clean.copy()
    filtered_df['Filtered_UV_1_280_mAU'] = uniform_filter1d(df_clean['UV_1_280_mAU
        '].values, size=15)
    filtered_df['Filtered_UV_2_260_mAU'] = uniform_filter1d(df_clean['UV_2_260_mAU
        '].values, size=15)

    # Step 3: Calculate rolling standard deviation using np.convolve for speed
    kernel_size = 15
    kernel = np.ones(kernel_size) / kernel_size
    kernel_sq = kernel ** 2

    sum_sq = np.convolve(filtered_df['Filtered_UV_1_280_mAU'].values, kernel_sq,
        mode='same')
    sum_sq_2 = np.convolve(filtered_df['Filtered_UV_2_260_mAU'].values, kernel_sq,
         mode='same')

    mean_sq_1 = np.convolve(filtered_df['Filtered_UV_1_280_mAU'].values, kernel,
        mode='same') ** 2
    mean_sq_2 = np.convolve(filtered_df['Filtered_UV_2_260_mAU'].values, kernel,
        mode='same') ** 2

    var_1 = sum_sq / kernel_size - mean_sq_1
    var_2 = sum_sq_2 / kernel_size - mean_sq_2

    var_1 = np.maximum(var_1, 0)
    var_2 = np.maximum(var_2, 0)

    filtered_df['RollingStd_UV_1_280_mAU'] = np.sqrt(var_1)
    filtered_df['RollingStd_UV_2_260_mAU'] = np.sqrt(var_2)

    # Step 4: Validate Unnamed: 0
    original_index = df_clean['Unnamed: 0'].values
```

```
is_sequential = np.all(np.diff(original_index) == 1)
if not is_sequential:
    filtered_df = filtered_df.reset_index(drop=True)

# Step 5: Calculate statistics
raw_stats = df_clean[uv_cols].agg(['mean', 'std'])
filtered_stats = filtered_df[uv_cols].agg(['mean', 'std'])

# Identify minimum baseline floor
min_baseline = filtered_df[uv_cols].min().min()

# Average rolling std dev
avg_rolling_std = filtered_df[['RollingStd_UV_1_280_mAU', '
    RollingStd_UV_2_260_mAU']].mean()

# Check if filtered_df is empty
if len(filtered_df) == 0:
    print("Warning: Filtered dataframe is empty. Skipping statistics
        calculation.")
    avg_rolling_std = pd.DataFrame()
    min_baseline = np.nan
    filtered_stats = pd.DataFrame()
    raw_stats = pd.DataFrame()

# Generate Markdown Table
output_lines = []
output_lines.append("## Mean/Std Dev: Raw vs. Filtered")
output_lines.append("| Column | Mean (Raw) | Std (Raw) | Mean (Filtered) | Std
    (Filtered) |")
output_lines.append("
    |--------|------------|-----------|-----------------|----------------|")
for col in uv_cols:
    m_raw, s_raw = raw_stats[col]
    m_filt, s_filt = filtered_stats[col]
    output_lines.append(f"| {col} | {m_raw:.4f} | {s_raw:.4f} | {m_filt:.4f} |
        {s_filt:.4f} |")

output_lines.append("")
output_lines.append("## Identified Minimum Baseline Floor")
output_lines.append("| Metric | Value |")
output_lines.append("|--------|-------|")
output_lines.append(f"| Min Baseline | {min_baseline:.4f} |")
output_lines.append("")

output_lines.append("## Average Rolling Std Dev")
output_lines.append("| Column | Avg Rolling Std Dev |")
output_lines.append("|--------|---------------------|")
output_lines.append(f"| UV_1_280_mAU | {avg_rolling_std['
    RollingStd_UV_1_280_mAU']:.4f} |")
output_lines.append(f"| UV_2_260_mAU | {avg_rolling_std['
    RollingStd_UV_2_260_mAU']:.4f} |")

# Line count check
if len(output_lines) > 20:
    print(f"Warning: Output exceeds 20 lines ({len(output_lines)} lines).
        Truncating.")
    output_lines = output_lines[:20]

output_text = "\n".join(output_lines)
```

```
    with open('/workspace/analysis_results_step1.md', 'w') as f:
        f.write(output_text)

    print(output_text)

if __name__ == "__main__":
    call_code_updates()
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import savgol_filter

def load_data(file_path):
    df = pd.read_csv(file_path)
    return df

def apply_median_filter(signal, kernel_size=3):
    return savgol_filter(signal, kernel_size, 2)

def calculate_dynamic_threshold(signal, baseline_floor, window_size=5, multiplier
    =3.5):
    rolling_std = np.convolve(signal, np.ones(window_size)/window_size, mode='same
        ')
    threshold = baseline_floor + multiplier * rolling_std
    return threshold

def identify_peaks(signal, threshold):
    above = signal > threshold
    below = signal <= threshold
    start_indices = np.where(np.diff(above) == -1)[0] + 1
    end_indices = np.where(np.diff(below) == -1)[0]
    peak_indices = []
    for i in range(len(start_indices)):
        if i < len(end_indices):
            peak_indices.append((start_indices[i], end_indices[i]))
    return peak_indices

def plot_analysis(df, output_path):
    if df.empty:
        print("Error: DataFrame is empty.")
        return

    if 'run' not in df.columns:
        print("Error: 'run' column not found in dataframe.")
        return

    runs = df['run'].unique()
    if not np.issubdtype(runs.dtype, np.integer):
        df = df.reset_index(drop=True)
        runs = df['run'].unique()

    fig, axes = plt.subplots(len(runs), 1, figsize=(14, 3 * len(runs)))
    if len(runs) == 1:
        axes = np.array([[axes]])

    for idx, run in enumerate(runs):
```

```
        run_data = df[df['run'] == run].sort_values('run')
        index = run_data.index.values
        signal = run_data['UV_1_280_mAU'].values

        if len(signal) == 0:
            continue

        filtered_signal = apply_median_filter(signal)
        threshold = calculate_dynamic_threshold(filtered_signal, np.min(
            filtered_signal))
        peaks = identify_peaks(signal, threshold)

        ax = axes[idx]
        ax.plot(index, signal, label='Raw', alpha=0.5)
        ax.plot(index, filtered_signal, label='Median-Filtered', linewidth=2)
        ax.plot(index, threshold, label='Dynamic Threshold', linestyle='--', color
            ='red', alpha=0.7)

        for start, end in peaks:
            ax.axvspan(start, end, color='green', alpha=0.2)

        ax.set_title(f'Run {run}')
        ax.set_xlabel('Index')
        ax.set_ylabel('mAU')
        ax.legend()
        ax.grid(True, alpha=0.3)
        ax.set_xlim(0, len(index))

    plt.suptitle('Dynamic Threshold and Spike Removal')
    plt.tight_layout()
    plt.savefig(output_path, dpi=300)
    plt.close()

# Main execution
file_path = '/inputs/chromatography_combined.csv'
output_path = '/workspace/dynamic_threshold_analysis.png'

df = load_data(file_path)
plot_analysis(df, output_path)
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Define variables explicitly
uv1 = df['UV_1_280_mAU']
uv2 = df['UV_2_260_mAU']

# Guard clause for empty DataFrame
if df.empty:
    print("Error: Input DataFrame is empty.")
    exit()

# Calculate window_size dynamically based on UV_1_280_mAU rolling std (calculated
    once)
```

```
window_size = int(uv1.rolling(window=50).std().mean())
if window_size == 0:
    window_size = 50

# Calculate rolling std for sorted data
if 'run' in df.columns:
    # Sort once and reuse sorted variables
    df_sorted = df.sort_values('run').reset_index(drop=True)
    uv1_sorted = df_sorted['UV_1_280_mAU']
    uv2_sorted = df_sorted['UV_2_260_mAU']

    # Define rolling_std_sorted
    rolling_std_sorted = uv1_sorted.rolling(window=window_size).std()

    # Define anchored_floor_sorted
    anchored_floor_sorted = uv1_sorted.median()

    # Define snr_sorted
    snr_sorted = (uv1_sorted - anchored_floor_sorted) / rolling_std_sorted

    # Define threshold_sorted
    threshold_sorted = anchored_floor_sorted + 3 * rolling_std_sorted

    # Define negative_impact string
    negative_impact = "Negative␣baseline␣reduces␣sensitivity␣by␣increasing␣noise␣
        floor."

    # Define avg_snr_uv1 and avg_snr_uv2 BEFORE the NaN check
    avg_snr_uv1 = snr_sorted.mean()
    avg_snr_uv2 = ((uv2_sorted - anchored_floor_sorted) / rolling_std_sorted).mean
        ()

    # Define fp_rate
    fp_rate = ((uv1_sorted > threshold_sorted) & (uv2_sorted > threshold_sorted)).
        sum() / len(uv1_sorted) * 100

    # Define robustness string
    if np.isnan(anchored_floor_sorted) or np.isnan(rolling_std_sorted).any():
        robustness = "Insufficient␣data␣for␣baseline␣calculation."
    else:
        robustness = f"Negative␣baseline␣reduces␣sensitivity.␣Avg␣SNR:␣{
            avg_snr_uv1:.2f},␣{avg_snr_uv2:.2f}.␣FP␣Rate:␣{fp_rate:.2f}%."
else:
    # Fallback if 'run' column is missing
    uv1_sorted = uv1
    uv2_sorted = uv2

    rolling_std_sorted = uv1_sorted.rolling(window=window_size).std()
    anchored_floor_sorted = uv1_sorted.median()

    snr_sorted = (uv1_sorted - anchored_floor_sorted) / rolling_std_sorted
    threshold_sorted = anchored_floor_sorted + 3 * rolling_std_sorted

    negative_impact = "Negative␣baseline␣reduces␣sensitivity␣by␣increasing␣noise␣
        floor."

    # Define avg_snr_uv1 and avg_snr_uv2 BEFORE the NaN check
    avg_snr_uv1 = snr_sorted.mean()
    avg_snr_uv2 = ((uv2_sorted - anchored_floor_sorted) / rolling_std_sorted).mean
```

```
        ()

    fp_rate = ((uv1_sorted > threshold_sorted) & (uv2_sorted > threshold_sorted)).
        sum() / len(uv1_sorted) * 100

    if np.isnan(anchored_floor_sorted) or np.isnan(rolling_std_sorted).any():
        robustness = "Insufficient data for baseline calculation."
    else:
        robustness = f"Negative baseline reduces sensitivity. Avg SNR: {
            avg_snr_uv1:.2f}, {avg_snr_uv2:.2f}. FP Rate: {fp_rate:.2f}%."

# Generate summary report
report = f"""
Signal-to-Noise & False Positive Report:
1. Avg SNR (UV1): {avg_snr_uv1:.2f}
2. Avg SNR (UV2): {avg_snr_uv2:.2f}
3. Est. False Positive Rate: {fp_rate:.2f}%
4. Robustness: {robustness}
"""

# Print report
print(report)
```

Listing 3: Code_3